Laporan Proyek Berpikir Komputasional (WI1102)

Kelas K05



Disusun Oleh:

Jovan Nathaniel Permana (16026295)
Qamila Annur Firdausi (16026380)
Abira Mutia Salsabila (16026060)
Ezra Hadyan Wicaksono (16026055)

Fakultas FMIPA Institut Teknologi Bandung

Semester 1 Tahun 2026/2027

1 Dasar-dasar Berpikir Komputasional yang Digunakan

1.1 Algoritma Newton-Raphson

1.1.1 Dekomposisi

Algoritma Newton-Raphson sebenarnya adalah algoritma optimasi yang bersifat iteratif, dengan tujuan utama untuk mencari akar-akar/titik potong sebuah fungsi $f(x)$ dengan sumbu x. Dalam proyek ini, algoritma Newton-Raphson difokuskan untuk mencari akar-akar dari sebuah polinom dengan derajat tertentu, sesuai masukan dari pengguna (*user*). Melalui prinsip dekomposisi (pemecahan suatu masalah dengan skala besar menjadi skala yang lebih kecil), algoritma Newton-Raphson untuk mencari akar-akar sebuah fungsi polinom dapat dipecah menjadi beberapa tugas berikut.

1. Menginput derajat polinom (N), nilai koefisien-koefisiennya, serta nilai tebakan awal (x_0) dari user.
2. Untuk nilai tebakan awal $x = x_0$ yang diberikan, hitung nilai fungsi $f(x_0)$ beserta turunannya, yaitu $f'(x_0)$.
3. Nilai x diperbarui secara terus-menerus melalui prosedur iteratif dengan persamaan
$$x_{n+1} \leftarrow x_n - \frac{f(x_n)}{f'(x_n)} \tag{1}$$
hingga nilai *error* yang terjadi dalam penentuan akar menjadi tidak signifikan atau berada di bawah ambang batas tertentu.
4. Jika $f'(x) = 0$ pada suatu tahap tertentu atau perulangan pada langkah 3 tidak dapat selesai (*infinite loop*), maka proses perhitungan akar gagal dilakukan.

1.1.2 Pengenalan Pola

Algoritma Newton-Raphson memiliki pola yang mirip dengan algoritma optimasi, yaitu algoritma yang menekankan perhitungan matematika secara iteratif (berulang) untuk mencapai suatu tujuan. Dalam algoritma Newton-Raphson, persamaan (1) dijalankan secara berulang-ulang untuk mencapai suatu tujuan, yaitu akar persamaan polinom. Selama perulangan tersebut, nilai x akan terus diperbarui (*update*) hingga error dalam pencarian akar menjadi tidak signifikan atau lebih kecil dari ambang batas tertentu.

1.1.3 Abstraksi

Algoritma Newton-Raphson membutuhkan beberapa informasi penting, yaitu:

1. Derajat polinom N.
2. Koefisien-koefisien pada polinom.
3. Nilai tebakan awal x_0.

Dalam proyek ini, derajat polinom N disimpan di dalam variabel bertipe `int`, sementara nilai tebakan awal x_0 disimpan di dalam variabel bertipe `float`. Nilai-nilai koefisien pada polinom disimpan di dalam sebuah array `float` yang berukuran $N + 1$.

1.2 Algoritma Pinjam Buku

1.2.1 Dekomposisi

1.2.2 Pengenalan Pola

1.2.3 Abstraksi

1.3 Algoritma Labirin

1.3.1 Dekomposisi

1.3.2 Pengenalan Pola

1.3.3 Abstraksi

2 Lampiran

2.1 Lampiran 1: Kode Python Tanpa Menggunakan Subprogram

2.1.1 Algoritma Newton-Raphson

Kode 1: Kode Python algoritma Newton-Raphson tanpa subprogram

```
1 import time
2 """ Newton Raphson (tanpa subprogram) """
3
4 # program utama
5 # input
```

```
N = int(input('Masukkan derajat polinomial: '))
N += 1
coefs = [0]*N
for i in range(N):
    if(i == 0):
        val = float(input('Masukkan nilai konstanta: '))
    else:
        val = float(input(f'Masukkan nilai koefisien x^{i}: '))

    coefs[i] = val

x = float(input('Masukkan tebakan awal x: '))
epsilon = 1e-4
k = 0
max_iter = 1e6
# proses iterasi
start_time = time.perf_counter()
while True:
    k += 1
    val = 0.0
    el = 0
    element = 1
    while el <= (N-1):
        val += coefs[el] * (x ** el)
        el += 1
    grad = 0.0
    while element <= (N-1):
        grad += element * coefs[element] * (x ** (element - 1))
        element += 1

    if grad == 0:
        print('Newton-Raphson gagal: turunan = 0')
        break

    x_new = x - (val / grad)
    print(f'Iterasi ke-{k} --> Nilai x: {x_new:.6f} Gradien: {grad:.6f}')
    delta = x_new - x
    if delta < 0:
        delta = -delta

```

```
    if val > 0:
        absval = val
    else:
        absval = -val

    if (delta < epsilon) and (absval < epsilon):
        print(f'Akar ditemukan: {x_new:.6f}')
        print(f'Jumlah iterasi: {k}')
        break
    elif k >= max_iter:
        print(f'\nGAGAL: Akar tidak ditemukan setelah {k} iterasi.')
        break

    x = x_new

end_time = time.perf_counter()
execution_time = end_time - start_time
print(f"Waktu eksekusi: {execution_time:.6f} detik")
```

2.1.2 Algoritma Pinjam Buku

Kode 2: Kode Python algoritma Pinjam Buku tanpa subprogram

```
"""Pinjam Buku (Tanpa Subprogram)"""
```

2.1.3 Algoritma Labirin

Kode 3: Kode Python algoritma Labirin tanpa subprogram

```
"""labirin (tanpa Subprogram)"""
# input
N = int(input("Masukkan ukuran labirin: "))
matrix = []
for i in range(N):
    col = [0]*N
    matrix += [col]

for y in range(N):
    for x in range(N):
        val = int(input(f'Apakah koordinat ({x},{y}) bisa dilewati (1) atau tidak (0): '))
```

```
        matrix[y][x] = val

# proses
# inisialisasi pointer dan arahnya
status_sisi = [[-1,-1], [-1,-1], [-1,-1], [-1,-1]]
# cari titik awal di sisi atas
for i in range(N):
    if matrix[0][i] == 1:
        status_sisi[0] = [i,0]
        break
# cari titik awal di sisi bawah
for i in range(N):
    if matrix[N-1][i] == 1:
        status_sisi[1] = [i,N-1]
        break
# cari titik awal di sisi kiri
for i in range(N):
    if matrix[i][0] == 1:
        status_sisi[2] = [0,i]
        break
# cari titik awal di sisi kanan
for i in range(N):
    if matrix[i][N-1] == 1:
        status_sisi[3] = [N-1, i]
        break

# tentukan endpoint
endpoints = []
for i in range(4):
    if status_sisi[i] != [-1,-1]:
        endpoints += [status_sisi[i]]

# tentukan titik mulai
pointer = endpoints[0]
direction = [0,0,0,0]
# cek apakah titik mulai terletak di titik sudut persegi
    labirin
if (pointer[0] == 0) and (pointer[1] == 0):
    direction = [0,1,1,0]
elif (pointer[0] == 0) and (pointer[1] == N-1):
    direction = [1,1,0,0]
elif (pointer[0] == (N-1)) and (pointer[1] == 0):
    direction = [0,0,1,1]
```

```
54 elif (pointer[0] == (N-1)) and (pointer[1] == (N-1)):
55     direction = [1,0,0,1]
56 elif (0 < pointer[0] < (N-1)) and (pointer[1] == 0):
57     direction = [0,1,1,1]
58 elif (0 < pointer[0] < (N-1)) and (pointer[1] == (N-1)):
59     direction = [1,1,0,1]
60 elif (pointer[0] == 0) and (0 < pointer[1] < (N-1)):
61     direction = [1,1,1,0]
62 elif (pointer[0] == (N-1)) and (0 < pointer[1] < (N-1)):
63     direction = [1,0,1,1]
64
65 while pointer != endpoints[1]:
66     for item in range(4):
67         if direction[item] == 1:
68             avail_dir = item
69             break
70     if avail_dir == 0:
71         pointer[1] += 1
72     elif avail_dir == 1:
73         pointer[0] += 1
74     elif avail_dir == 2:
75         pointer[1] -= 1
76     elif avail_dir == 3:
77         pointer[0] -= 1
78
79     print(f'Pointer melewati x: {pointer[0]}, y: {
       pointer[1]}')
80
81     if (pointer[0] == 0) and (pointer[1] == 0):
82         direction = [0,1,1,0]
83     elif (pointer[0] == 0) and (pointer[1] == N-1):
84         direction = [1,1,0,0]
85     elif (pointer[0] == (N-1)) and (pointer[1] == 0):
86         direction = [0,0,1,1]
87     elif (pointer[0] == (N-1)) and (pointer[1] == (N-1))
       :
88         direction = [1,0,0,1]
89     elif (0 < pointer[0] < (N-1)) and (pointer[1] == 0):
90         direction = [0,1,1,1]
91     elif (0 < pointer[0] < (N-1)) and (pointer[1] == (N
       -1)):
92         direction = [1,1,0,1]
93     elif (pointer[0] == 0) and (0 < pointer[1] < (N-1)):
```

```
94          direction = [1,1,1,0]
95      elif (pointer[0] == (N-1)) and (0 < pointer[1] < (N
            -1)):
96          direction = [1,0,1,1]
97      elif matrix[pointer[1]+1][pointer[0]] == 0:
98          direction = [0,1,1,1]
99      elif matrix[pointer[1]-1][pointer[0]] == 0:
100         direction = [1,1,0,1]
101     elif matrix[pointer[1]][pointer[0]+1] == 0:
102         direction = [1,0,1,1]
103     elif matrix[pointer[1]][pointer[0]-1] == 0:
104         direction = [1,1,1,0]
105     else:
106         direction = [1,1,1,1]
```

2.2 Lampiran 2: Kode Python Dengan Menggunakan Subprogram

2.2.1 Algoritma Newton-Raphson

Kode 4: Kode Python algoritma Newton-Raphson dengan subprogram

```
1  import time
2  """ Newton Raphson (dengan subprogram) """
3
4  # definisi fungsi
5  def function(x, coefs):
6      value = 0.0
7      for i in range(len(coefs)):
8          value += coefs[i] * (x**i)
9      return value
10
11 def gradient(x, coefs):
12     value = 0.0
13     for i in range(1, len(coefs)):
14         value += i * coefs[i] * (x**(i - 1))
15     return value
16
17 def absol(x):
18     value = (x >= 0) * x + (x < 0) * (-x)
19     return value
20
21 # program utama
```

```
# input
N = int(input('Masukkan derajat polinomial: '))
N += 1
coefs = [0]*N
for i in range(N):
    if(i == 0):
        val = float(input('Masukkan nilai konstanta: '))
    else:
        val = float(input(f'Masukkan nilai koefisien x^{i}: '))

    coefs[i] = val

x = float(input('Masukkan tebakan awal x: '))
epsilon = 1e-4
k = 0
max_iter = 1e6
# proses iterasi
start_time = time.perf_counter()
while True:
    k += 1
    val = function(x, coefs)
    grad = gradient(x, coefs)
    if grad == 0:
        print('Newton-Raphson gagal: turunan = 0')
        break

    x_new = x - (val / grad)
    print(f'Iterasi ke-{k} --> Nilai x: {x_new:.6f} Gradien: {grad:.6f}')

    if (absol(x_new - x) < epsilon) and (absol(val) < epsilon):
        print(f'Akar ditemukan: {x_new:.6f}')
        print(f'Jumlah iterasi: {k}')
        break
    elif k >= max_iter:
        print(f'\nGAGAL: Akar tidak ditemukan setelah {k} iterasi.')
        break

    x = x_new

```

```
61 end_time = time.perf_counter()
62 execution_time = end_time - start_time
63 print(f"Waktu eksekusi: {execution_time:.6f} detik")
```

2.2.2 Algoritma Pinjam Buku

Kode 5: Kode Python algoritma Pinjam Buku dengan subprogram

```
1 """Pinjam Buku (dengan Subprogram)"""
```

2.2.3 Algoritma Labirin

Kode 6: Kode Python algoritma Labirin dengan subprogram

```
1 """labirin (dengan Subprogram)"""
```